Authorization Sync Between Unity Catalog and Neo4j

Bridging two privilege models through a common semantic layer

Neo4j JDBC Lakehouse Federation: Working Prototype

- **Prototype validated:** Neo4j JDBC registered as a UC JDBC connector, working end-to-end in Databricks
- **Neo4j JDBC driver enables SQL-to-Cypher translation:** `SELECT COUNT(*) FROM Aircraft` becomes `MATCH (n:Aircraft) RETURN count(n)` automatically
- **Federated queries:** Neo4j graph data joins with Delta tables in a single Spark SQL statement
- **Current issue:** custom connectors run in an isolated sandbox, adding 13-16 seconds of overhead per query

Status: Reviewing with Databricks

- **Passed technical validation** with the Databricks Partner Solution Architect
- **Waiting on product management** approval to list Neo4j as an officially supported data source
- **Native status would place Neo4j** alongside PostgreSQL, Snowflake, MySQL, BigQuery, and other supported sources
- **Customer demand:** Boeing, Syngenta, IBM Federal, Gilead, and multiple Databricks Federal accounts

What Official Federated Data Source Status Unlocks

- **Foreign catalog creation:** `CREATE FOREIGN CATALOG` registers Neo4j as a browsable three-level namespace in Unity Catalog
- **Table-level permissions:** UC grants (`SELECT`, `USE CATALOG`, `BROWSE`) on individual Neo4j-backed tables, not just connection-level access
- **Managed OAuth:** built-in IdP flows (Entra ID, Okta) instead of manually embedding credentials
- **Per-table audit logging:** every query appears in `system.access.audit` with full context
- **Genie natural language queries:** users ask about graph data in plain English without materialized Delta copies
- **Optimized query pushdown:** filters, aggregates, sorts, and joins pushed down as Cypher rather than generic JDBC

UC Table-Level Access Controls: Partial Coverage

- **When queries flow through DBSQL:** UC enforces `SELECT`, column masks, and row filters on federated Neo4j tables
- **An analyst querying `neo4j_catalog.default.person`** sees only the columns and rows their UC role permits
- **But this only covers the SQL path:** access controls apply at the Databricks layer, not inside Neo4j
- **Direct Neo4j access bypasses UC entirely:** a user connecting via Bolt or Cypher Shell sees whatever their Neo4j role permits, regardless of UC grants

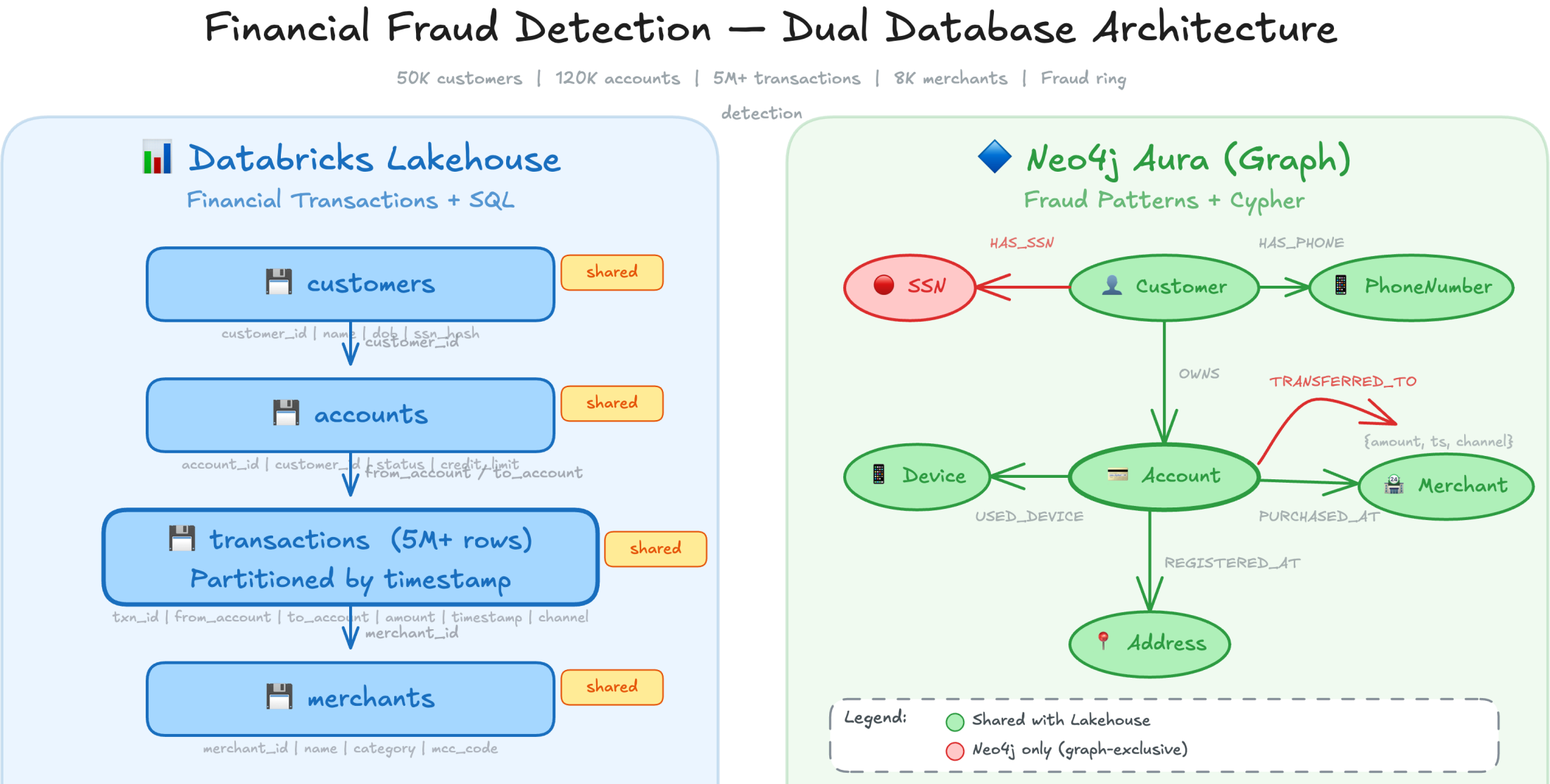
Beyond the SQL Path

What happens when access doesn't flow through Unity Catalog

The Governance Gap

- **Two systems, overlapping data:** UC governs tables, Neo4j governs graph traversal
- **No shared understanding:** a `Person` table and a `:Person` label represent the same concept through different structures
- **Permission granted in one, absent in the other:** users blocked from legitimate work or accessing data they shouldn't
- **Different access semantics:** `SELECT` on rows is not the same as `MATCH` on nodes

The Dual Database Architecture



What Is a Semantic Layer?

- **Declarative description** of what your data means, placed on top of the data
- **Any consumer shares** a common understanding of how data should be interpreted
- **Two levels of abstraction:** concept layer (business meaning) and metadata layer (physical structures)
- **Built as a graph** connecting business concepts to UC tables and Neo4j labels

The Semantic Layer Data Model

```
(:Concept {name: "Portfolio"})
    -[:MEASURED_BY]-> (:Metric)
        -[:SOURCED_FROM]-> (:Column)
            <-[:HAS_COLUMN]- (:Table {schema: "neo4j_catalog"})

(:Concept {name: "Portfolio"})
    -[:MAPS_TO]-> (:Label {name: "Portfolio"})   // Neo4j side
```

- **Concept to UC:** Portfolio connects through metrics and columns to `fact_positions`, `dim_account`, `dim_customer`
- **Concept to Neo4j:** Portfolio maps directly to the `:Portfolio` label
- **One definition, two systems:** the graph already knows what "Portfolio" means in both

Proposed Authorization Sync Patterns

Four approaches to aligning privileges across both systems

Different tradeoffs for consistency, operational overhead, and privilege granularity

Pattern 1: Shared Identity Provider

- **Single IdP for both systems:** Okta, Microsoft Entra ID, or Google
- **Group membership drives role assignment** in both systems independently
- **No running sync job:** both systems authenticate against the same provider
- **One-time configuration:** Neo4j admin defines roles and Cypher privileges per IdP group

Pattern 1: UC to Neo4j Privilege Mapping

UC Concept	Neo4j Equivalent	Example
Table-level `SELECT`	Label-level `MATCH`	`GRANT MATCH {*} ON GRAPH * NODES Person TO analyst`
Column projection	Property-level `READ`	`GRANT READ {name, email} ON GRAPH * NODES Person TO analyst`
Row filter	Property-based access control	`GRANT READ {*} ... FOR (n:Account) WHERE n.region = 'EU'`
Column mask	Property-level `DENY READ`	`DENY READ {ssn} ON GRAPH * NODES Person TO general_users`

Pattern 1: The Mapping Problem

- **A UC table is not a Neo4j label:** `SELECT` returns rows, `MATCH` returns nodes that participate in traversals
- **Relationships have no UC equivalent:** `TRAVERSE` on `:WORKS_AT` controls edge traversal, UC has no concept of this
- **PBAC has no table-side analog:** Neo4j restricts by property value, UC row filters use different syntax and semantics
- **Graph traversal is compositional:** revoking one label mid-chain changes query behavior in non-obvious ways

Pattern 2: Shared IdP Plus Semantic Layer

- **Access policies defined at the business concept level**, not per-system
- **Semantic layer encodes the mapping** between business concepts and both privilege models
- **Sync process traverses the graph** to derive system-specific privileges automatically
- **Schema changes propagate** through existing `GOVERNED_BY` relationships

Pattern 2: The Access Policy Model

```
// Define an access policy on a business concept
CREATE (ap:AccessPolicy {name: "portfolio-read", effect: "ALLOW"})

// Link policy to the business concept it governs
MATCH (c:Concept {name: "Portfolio"}), (ap:AccessPolicy {name: "portfolio-read"})
CREATE (c)-[:GOVERNED_BY]->(ap)

// Link policy to the IdP group that holds it
CREATE (g:Group {name: "graph-analysts", provider: "okta"})
CREATE (ap)-[:GRANTED_TO]->(g)
```

Pattern 2: Deriving Privileges from the Graph

```
// Derive UC tables that require SELECT for a given access policy
MATCH (ap:AccessPolicy {name: "portfolio-read"})<-[:GOVERNED_BY]-(c:Concept)
MATCH (c)-[:MEASURED_BY]->(m:Metric)-[:SOURCED_FROM]->(col:Column)
      <-[:HAS_COLUMN]-(t:Table)
RETURN DISTINCT t.name AS uc_table, COLLECT(col.name) AS columns

// Derive Neo4j label privileges for the same policy
MATCH (ap:AccessPolicy {name: "portfolio-read"})<-[:GOVERNED_BY]-(c:Concept)
RETURN c.name AS neo4j_label
```

- **One policy definition** produces grants for both systems
- **Cross-boundary conflicts** surface as flagged edges, not silent misconfigurations

Pattern 3: UC as Source of Truth, Push to Neo4j

1. **Read UC privilege state:** query `INFORMATION_SCHEMA.TABLE_PRIVILEGES` for materialized Neo4j tables
2. **Map UC grants to Neo4j privileges:** `SELECT` on table `person` becomes `GRANT MATCH {*} ON GRAPH * NODES Person`
3. **Apply to Neo4j:** connect via Bolt, execute Cypher admin commands against the system database
4. **Detect drift:** compare Neo4j privilege state against desired state on each run, revoke stale grants

Pattern 3: The Sync Job

```
with GraphDatabase.driver(NEO4J_BOLT_URI, auth=(NEO4J_USERNAME, NEO4J_PASSWORD)) as driver:
    with driver.session(database="system") as session:
        session.run("CREATE ROLE uc_analyst IF NOT EXISTS")
        session.run("GRANT MATCH {*} ON GRAPH * NODES Person TO uc_analyst")
        session.run("GRANT MATCH {*} ON GRAPH * NODES Company TO uc_analyst")
        session.run("DENY READ {ssn} ON GRAPH * NODES Person TO uc_analyst")
```

- **Eventually consistent:** bounded by the job schedule
- **Fits when:** UC is the established governance layer and Neo4j is downstream

Pattern 4: Neo4j as Source of Truth, Push to UC

1. **Read Neo4j privilege state:** execute `SHOW PRIVILEGES` via Cypher for `TRAVERSE`, `READ`, `MATCH`
2. **Map Neo4j privileges to UC grants:** `MATCH` on `:Person` becomes `GRANT SELECT ON TABLE neo4j_catalog.default.person`
3. **Apply to UC:** execute `GRANT` / `REVOKE` via Statement Execution API or SQL warehouse connection
4. **Column masks for property denials:** `DENY READ {ssn}` becomes `ALTER TABLE ... ALTER COLUMN ... SET MASK`

Choosing a Pattern

Consideration	Pattern 1: Shared IdP	Pattern 2: IdP + Semantic Layer	Pattern 3: UC to Neo4j	Pattern 4: Neo4j to UC
Sync job required	No	Yes	Yes	Yes
Source of truth	Identity provider	Semantic layer graph	Unity Catalog	Neo4j
Consistency	Immediate (on auth)	Eventually consistent	Eventually consistent	Eventually consistent
Property-level control	Manual Neo4j config	Derived from semantic layer	Via UC column masks (lossy)	Via generated UC column masks

The External Metadata API: Visibility Without Enforcement

- **Registers Neo4j labels and relationships** as metadata objects in Unity Catalog
- **Supports** `BROWSE`, `MODIFY`, `MANAGE`: controls who can see and manage the metadata registration
- **Does not propagate to Neo4j:** granting `BROWSE` on a `:Person` metadata object does not grant `TRAVERSE` on `:Person` nodes
- **No permission change events:** both systems require polling, neither offers push-based webhooks

Authorization Sync: The Combined Picture

- **Pattern 1 (Shared IdP):** simplest, no sync job, but manual mapping and coarse-grained
- **Pattern 2 (Semantic Layer):** graph-derived privileges, handles relationships and PBAC, requires semantic layer maintenance
- **Pattern 3 (UC to Neo4j):** UC-governed environments, scheduled push, lossy on column masks
- **Pattern 4 (Neo4j to UC):** graph-governed environments, scheduled push, lossy on PBAC
- **Semantic layer** relocates complexity from admin reasoning to graph traversal